

Guion de Clase: Zero-Shot Classification con CLIP

Notebook: `class/CNN/CLIP_Zero_Shot_Classification.ipynb` **Prerequisito:** `class/NLP/Image_search`
Duración estimada: 1h 30min - 2h

Antes de empezar: Contexto desde Image Search

Los alumnos ya han trabajado con CLIP en el notebook de Image Search, donde:

- Usaron **sentence-transformers** para cargar CLIP (`SentenceTransformer('clip-ViT-B-32')`)
- Codificaron imágenes y textos en un espacio compartido de embeddings
- Implementaron búsqueda text-to-image e image-to-image con similitud coseno
- Trabajaron con el dataset Unsplash (25k imágenes)

En este notebook damos el salto de **buscar** imágenes a **clasificarlas**, usando CLIP directamente desde Hugging Face Transformers.

Hugging Face: El ecosistema de ML

Qué es Hugging Face

[Hugging Face](#) es la plataforma de referencia para compartir y usar modelos de machine learning. Funciona como un “GitHub de modelos”:

- **Model Hub** (huggingface.co/models): +500k modelos pre-entrenados (NLP, visión, audio, multimodal)
- **Datasets Hub** (huggingface.co/datasets): +100k datasets listos para usar
- **Spaces** (huggingface.co/spaces): demos interactivas de modelos (Gradio, Streamlit)

La librería principal es `transformers` (github.com/huggingface/transformers), que da acceso unificado a miles de modelos con una API consistente:

```
from transformers import CLIPModel, CLIPProcessor
```

```
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")  
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

Cada modelo en el Hub tiene una **model card** con documentación, métricas y ejemplos. Por ejemplo: huggingface.co/openai/clip-vit-base-patch32

Hugging Face Transformers vs Sentence Transformers

	Hugging Face transformers	sentence-transformers
Nivel	Bajo nivel, acceso completo al modelo	Alto nivel, orientado a embeddings

	Hugging Face transformers	sentence-transformers
API	<code>model.get_image_features()</code> <code>model(**inputs)</code>	<code>model.encode(images)</code> , <code>model.encode(texts)</code>
Control	Total: accedes a logits, capas intermedias, outputs crudos	Limitado: devuelve directamente el embedding normalizado
Preprocesado	Manual con processor	Automático
Uso típico	Clasificación, fine-tuning, investigación	Búsqueda semántica, similitud, clustering
Modelos	Todos los del Hub (+500k)	Subconjunto optimizado para embeddings (~5k)
Docs	huggingface.co/docs/transformers	sbert.net

En resumen: - En **Image Search** usamos sentence-transformers porque solo necesitábamos embeddings para buscar - En **este notebook** usamos transformers porque necesitamos acceso a los logits del modelo para calcular probabilidades de clasificación

Pipeline de Hugging Face (para mencionar brevemente)

Hugging Face también ofrece pipeline, una API de muy alto nivel para tareas comunes:

```
from transformers import pipeline
classifier = pipeline("zero-shot-image-classification", model="openai/clip-vit-base-pa
result = classifier(image, candidate_labels=["dog", "cat", "bird"])
```

Documentación: huggingface.co/docs/transformers/main_classes/pipelines

No lo usamos en el notebook porque queremos que los alumnos entiendan el mecanismo interno, pero es bueno que sepan que existe.

Guion sección por sección

1. Introduction (5 min)

Qué explicar: - Problema del clasificador tradicional: necesitas datos etiquetados, entrenamiento, y las clases son fijas - Zero-shot: describes las categorías en texto y el modelo clasifica sin entrenamiento - Conectar con lo que ya saben: “En Image Search usasteis CLIP para buscar; ahora lo vamos a usar para clasificar”

Punto clave: El modelo NO se reentrena. Usamos el mismo CLIP pre-entrenado, solo cambiamos cómo formulamos la tarea.

2. CLIP Architecture (5 min)

Qué explicar: - Recordar brevemente la arquitectura dual (image encoder + text encoder) que ya vieron en Image Search - Enfatizar que ambos encoders producen vectores en el **mismo espacio**

- Explicar el mecanismo de clasificación: codificar la imagen, codificar cada etiqueta candidata, calcular similitud coseno, la mayor similitud = clase predicha

Diferencia con Image Search: Allí buscábamos la imagen más similar a un texto. Aquí buscamos el texto (etiqueta) más similar a una imagen. Es la misma operación pero invertida.

3. Basic Zero-Shot Classification (10 min)

Qué explicar: - Diferencia de API respecto a Image Search: - Antes: `model.encode()` de `sentence-transformers` - Ahora: `CLIPProcessor` + `CLIPModel` de Hugging Face `transformers` - La función `classify_image()`: procesa imagen y textos juntos, obtiene `logits_per_image`, aplica softmax - Mostrar el ejemplo de la playa y el tiburón

Demo en vivo: Cambiar las etiquetas candidatas en directo. Por ejemplo, añadir “underwater” al ejemplo del tiburón y ver cómo cambian las probabilidades.

Open Vocabulary: Destacar que no hay lista fija de clases. Podemos poner cualquier descripción en texto natural. Mostrar el ejemplo de `dog_cat.jpg` con descripciones genéricas vs. muy específicas.

4. Prompt Engineering (10 min)

Qué explicar: - CLIP fue entrenado con pies de foto de internet, no con palabras sueltas - El template importa: “a photo of a dog” funciona mejor que “dog” - Mostrar la comparación de templates con el pájaro - Mencionar que esto es análogo al prompt engineering en LLMs

Punto para discusión: “¿Por qué creéis que a `blurry photo of a...` da peor resultado?”

Question 1 (10 min)

Los alumnos experimentan con templates para clasificar `traffic.jpg`. Deben probar al menos 3 templates diferentes.

Pista si se atascan: Probar templates con contexto de dominio como “a traffic camera photo showing {}” o “an aerial view of {}”.

5. Image-Text Similarity Space (10 min)

Qué explicar: - Aquí usamos directamente `get_image_features` y `get_text_features` (nivel más bajo que `classify_image`) - Construimos una matriz de similitud completa: cada imagen contra cada texto - La diagonal tiene los valores más altos: cada imagen encaja mejor con su descripción correspondiente - Conectar con Image Search: “Esto es exactamente lo que hacíais cuando buscabais imágenes, pero ahora lo visualizamos como matriz”

Nota técnica: Aquí usamos `pixel_values` y `input_ids` explícitamente en vez de `**kwargs` para compatibilidad con versiones recientes de `transformers`.

Question 2 (10 min)

Los alumnos escriben 3 queries creativas y ven qué imagen encaja mejor. Fomentar creatividad: queries abstractas como “danger”, “relaxation”, “weekend plans”.

Pregunta para discusión: “¿Alguna query ha dado un resultado sorprendente? ¿Por qué creéis que pasa?”

6. Zero-Shot on a Real Dataset (15 min)

Qué explicar: - Evaluamos CLIP en el dataset de flores que ya usaron en Introduction to CNN - Sin entrenamiento vs. CNN entrenada: comparar accuracy - La confusion matrix muestra dónde se equivoca CLIP

Punto clave para la discusión: - CNN from scratch: ~72% (necesitó datos + entrenamiento) - MobileNetV2 transfer learning: ~90% (necesitó datos + fine-tuning) - CLIP zero-shot: ~70-85% (sin datos, sin entrenamiento)

“CLIP consigue resultados competitivos sin haber visto nunca el dataset de flores. Eso es lo revolucionario del zero-shot.”

Question 3 (15 min)

Los alumnos intentan mejorar la accuracy con prompt engineering: - Mejores templates - Ensemble de templates (promediar predicciones) - Nombres de clase más descriptivos

Objetivo: Entender que el rendimiento zero-shot depende mucho de cómo formulas la tarea.

7. Beyond Object Categories (10 min)

Qué explicar: - CLIP no solo reconoce objetos, entiende conceptos abstractos - Mood classification: clasificar el “estado de ánimo” de una escena - Multi-attribute: extraer múltiples propiedades de una imagen (actividad, escenario, hora del día)

Punto clave: “Un clasificador tradicional solo puede responder la pregunta para la que fue entrenado. CLIP puede responder cualquier pregunta que formule en texto.”

Question 4 (10 min)

Los alumnos prueban CLIP con imágenes médicas (radiografía de tórax). Deben descubrir que: - CLIP reconoce que es una imagen médica (clasificación genérica funciona) - Pero falla en diagnósticos específicos (no distingue bien entre sano/neumonía/cáncer)

Discusión importante: Limitaciones del zero-shot en dominios especializados. CLIP fue entrenado con imágenes de internet, no con datos médicos. Para aplicaciones médicas reales necesitaríamos fine-tuning con datos específicos (y validación clínica).

8. What's Next (5 min)

Mencionar brevemente: - **Open Vocabulary Detection** (siguiente notebook): detectar objetos por descripción de texto - **OpenCLIP y SigLIP**: versiones mejoradas de CLIP - **Fine-tuning**: cómo adaptar CLIP a dominios específicos - **Generación de imágenes**: CLIP como componente de Stable Diffusion

Resumen de conceptos clave

Concepto	Dónde se cubre
Zero-shot classification	Secciones 1-3
HuggingFace Transformers vs sentence-transformers	Introducción (este guion)
Prompt engineering	Sección 4, Question 1 y 3
Espacio de embeddings compartido	Sección 5
Evaluación en dataset real	Sección 6
Limitaciones del zero-shot	Question 4
Clasificación multi-atributo	Sección 7

Links de referencia

- **CLIP paper:** arxiv.org/abs/2103.00020
- **Hugging Face Hub:** huggingface.co
- **Hugging Face Transformers docs:** huggingface.co/docs/transformers
- **CLIP model card:** huggingface.co/openai/clip-vit-base-patch32
- **Sentence Transformers docs:** [sbnet.net](https://www.sbert.net)
- **OpenCLIP:** github.com/mlfoundations/open_clip
- **SigLIP:** huggingface.co/google/siglip-base-patch16-224
- **Hugging Face Pipelines:** huggingface.co/docs/transformers/main_classes/pipelines